Journal homepage http://revistas.unitru.edu.pe/index.php/SSMM

SELECCIONES MATEMÁTICAS
Universidad Nacional de Trujillo
ISSN: 2411-1783 (Online)
2026; Vol. TBD: 1-20.

Solving the Many-Electron Schrödinger Equation With a Transformer Architecture

Jorge Muñoz Laredo

Received, *Accepted,*

How to cite this article:

Special Issue: Workshop on Optimization and Operator Learning - SaC3 - Perú

Abstract

The many-electron Schrödinger equation is a central problem in ab initio *quantum chemistry. Variational Monte Carlo (VMC) with flexible neural ansätze, as in FermiNet and related architectures, has shown that deep models can approximate ground-state wavefunctions accurately. This work presents a PyTorch-based study of a Slater–Jastrow ansatz in which a transformer-style backbone parameterizes single-particle orbitals entering a Slater determinant, together with a low-parameter Jastrow factor enforcing electron–electron cusp structure. We summarize the governing equations, the VMC objective and Metropolis–Hastings sampling, implementation choices for stable derivatives of the log-determinant via SVD, and batched energy evaluation for high GPU utilization. Representative convergence curves and energy estimates illustrate the behavior of the resulting* Psiformer-*style model on small molecular benchmarks.*

Keywords. Many-electron Schrödinger equation, variational Monte Carlo, transformer, neural wavefunction, Slater determinant, quantum chemistry.

1. Introduction. Computing the ground state of many-electron systems requires solving the electronic Schrödinger equation in the presence of Coulomb interactions and fermionic antisymmetry. Deep variational wavefunctions trained with VMC have emerged as a practical route to high-accuracy energies without an explicit basis expansion in the traditional sense [2, 1]. Self-attention mechanisms, as introduced for sequence modeling [3], motivate architectures that mix information across electrons while respecting permutation structure when combined with a Slater determinant. This article collects the material from an expository slide deck into article form and documents implementation aspects of a Psiformer-style ansatz in PyTorch.

2. The Schrödinger wave function and physical constraints.

2.1. Schrödinger equation. In 1926 Schrödinger formulated the time-dependent Schrödinger equation (TDSE),

$$i\,\hbar\,\partial_t \Psi = \hat{H}\,\Psi. \tag{2.1}$$

Here $\Psi \in \mathcal{H}$ is the **wave function**, $\hat{H}$ is the **Hamiltonian**, and the state depends on spatial coordinates and time. The quantity $|\Psi|^2$ is interpreted as a probability density (normalized to integrate to one).

In the position representation,

$$\hat{H} = \frac{\hat{\vec{P}}^2}{2m} + \hat{V} = -\frac{\hbar^2}{2m}\nabla^2 + \hat{V}. \tag{2.2}$$

*Instituto de Investigaciones, Universidad de Puebla, Puebla, México (slepe@upuebla.edu.pe)

Separating $\psi(\vec{\mathbf{r}}, t) = R(\vec{\mathbf{r}})T(t)$ leads to

$$T(t) = e^{-iEt/\hbar}, \qquad \hat{H}R(\vec{\mathbf{r}}) = ER(\vec{\mathbf{r}}), \tag{2.3}$$

with E the total energy. The stationary problem is to find ψ such that

$$\left[-\frac{\hbar^2}{2m}\nabla^2 + V(\mathbf{r}, t)\right]\psi(\vec{\mathbf{r}}) = E\psi(\vec{\mathbf{r}}), \tag{2.4}$$

given the potential V of the system.

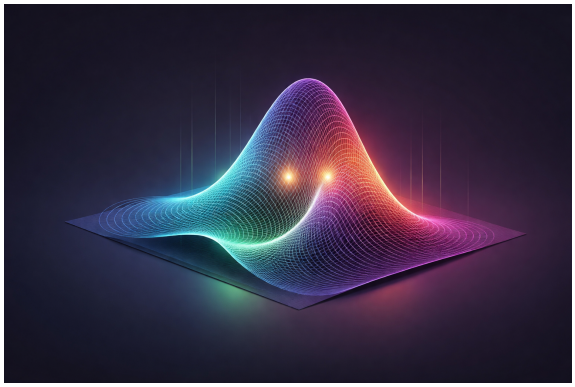


Figure 2.1: $|\Psi(\mathbf{R})|^2$ as the probability density for finding particles near configuration $\mathbf{R}$.

2.2. Many-body Hamiltonian. For n electrons (and clamped nuclei in the Born–Oppenheimer approximation), collective coordinates $\mathbf{x}_i = (\mathbf{r}_i, \sigma_i)$ combine position $\mathbf{r}_i \in \mathbb{R}^3$ and spin $\sigma_i \in \{\uparrow, \downarrow\}$. The eigenproblem reads

$$\hat{H}\psi(\mathbf{x}_1, \ldots, \mathbf{x}_n) = E\psi(\mathbf{x}_1, \ldots, \mathbf{x}_n). \tag{2.5}$$

Each particle interacts with all others; for atoms one would include nuclei and electrons consistently, subject to the chosen level of approximation.

A standard molecular Hamiltonian (atomic units) takes the form

$$\begin{aligned}\hat{H} = &-\sum_{i=1}^{N}\frac{1}{2}\nabla_i^2 - \sum_{I=1}^{M}\frac{1}{2M_I}\nabla_I^2 - \sum_{i=1}^{N}\sum_{I=1}^{M}\frac{Z_I}{|\mathbf{r}_i - \mathbf{R}_I|} \\ &+ \sum_{1\le i<j\le N}\frac{1}{|\mathbf{r}_i - \mathbf{r}_j|} + \sum_{1\le I<J\le M}\frac{Z_I Z_J}{|\mathbf{R}_I - \mathbf{R}_J|}.\end{aligned} \tag{2.6}$$

The Born–Oppenheimer approximation fixes nuclear positions and often drops the nuclear kinetic terms when solving for the electronic wavefunction.

2.3. Fermi–Dirac statistics and cusps. Electrons are indistinguishable fermions: exchanging two electrons flips the sign of Ψ, and the Pauli principle forbids double occupation of the same spin orbital. A Slater determinant built from spin orbitals $\phi_j^k(\mathbf{x}_i)$ enforces antisymmetry,

$$\psi = \begin{vmatrix} \phi_1^k(\mathbf{x}_1) & \ldots & \phi_1^k(\mathbf{x}_n) \\ \vdots & \ddots & \vdots \\ \phi_n^k(\mathbf{x}_1) & \ldots & \phi_n^k(\mathbf{x}_n) \end{vmatrix}. \tag{2.7}$$

Coulomb interactions induce cusps in Ψ when particles coalesce. Kato-type cusp conditions include electron–nucleus

$$\frac{\partial\Psi}{\partial r_{iI}}\Big|_{r_{iI}=0} = -\,Z_I\,\Psi(0)$$

and electron–electron

$$\frac{\partial\Psi}{\partial r_{ij}}\Big|_{r_{ij}=0} = \frac{1}{2}\,\Psi(0).$$

A Jastrow factor $\exp(\mathcal{J})$ is often used to impose short-range correlation structure. In this work we use

$$\mathcal{J}_\theta(\mathbf{x}) = \sum_{\substack{i<j \\ \sigma_i=\sigma_j}} -\frac{1}{4}\frac{\alpha_{\text{par}}^2}{\alpha_{\text{par}} + |\mathbf{r}_i - \mathbf{r}_j|} + \sum_{\substack{i,j \\ \sigma_i\neq\sigma_j}} -\frac{1}{2}\frac{\alpha_{\text{anti}}^2}{\alpha_{\text{anti}} + |\mathbf{r}_i - \mathbf{r}_j|}. \tag{2.8}$$

3. Variational Monte Carlo.

3.1. Variational principle and local energy. The variational principle states

$$E[\Psi] = \frac{\langle\Psi|\,\hat{H}\,|\Psi\rangle}{\langle\Psi|\Psi\rangle} \geq E_0. \tag{3.1}$$

Minimizing $E[\Psi]$ with respect to parameters drives the ansatz toward the ground state. Writing Ψ_θ for a parameterized trial function,

$$\mathcal{L}(\Psi_\theta) = \frac{\langle\Psi_\theta|\,\hat{H}\,|\Psi_\theta\rangle}{\langle\Psi_\theta|\Psi_\theta\rangle} = \frac{\int \Psi_\theta^*(\mathbf{R})\,\hat{H}\Psi_\theta(\mathbf{R})\,d\mathbf{R}}{\int \Psi_\theta^*(\mathbf{R})\Psi_\theta(\mathbf{R})\,d\mathbf{R}}. \tag{3.2}$$

With the normalized density $p_\theta(\mathbf{R}) \propto |\Psi_\theta(\mathbf{R})|^2$ and the *local energy*

$$E_L(\mathbf{R}) = \frac{\hat{H}\Psi_\theta(\mathbf{R})}{\Psi_\theta(\mathbf{R})}, \tag{3.3}$$

the loss is the expectation $\mathcal{L}_\theta = \mathbb{E}_{\mathbf{R}\sim p_\theta}[E_L(\mathbf{R})]$.

3.2. Monte Carlo estimation and sampling. Given samples $\mathbf{R}_1, \ldots, \mathbf{R}_M$ drawn from $|\Psi_\theta|^2$ (up to normalization),

$$\mathcal{L}_\theta = \mathbb{E}_{\mathbf{R}\sim|\Psi_\theta|^2}[E_L(\mathbf{R})] \approx \frac{1}{M}\sum_{k=1}^{M} E_L(\mathbf{R}_k). \tag{3.4}$$

The local energy can be written as

$$E_L(\mathbf{R}_k) = \frac{\hat{H}\psi(\mathbf{R}_k)}{\psi(\mathbf{R}_k)} = -\frac{1}{2}\frac{\nabla^2\psi(\mathbf{R}_k)}{\psi(\mathbf{R}_k)} + V(\mathbf{R}_k). \tag{3.5}$$

Samples are generated with Metropolis–Hastings: starting from $\mathbf{R}_0$, propose $\mathbf{R}' = \mathbf{R}_0 + \eta$ with $\eta \sim q$, accept with probability

$$A(\mathbf{R}_0, \mathbf{R}') = \min\left(1, \frac{\rho(\mathbf{R}')}{\rho(\mathbf{R}_0)}\right), \tag{3.6}$$

where $\rho \propto |\Psi_\theta|^2$, and iterate. Gradients of the VMC objective follow as (schematically)

$$\nabla_\theta\mathcal{L} = 2\,\mathbb{E}_{\mathbf{x}\sim|\Psi|^2}\left[(E_L(\mathbf{x}) - \mathbb{E}_p[E_L])\,\nabla_\theta \log\Psi_\theta(\mathbf{x})\right], \tag{3.7}$$

estimated with the same samples.

4. Transformer backbone and self-attention. Figure 4.1 sketches a transformer-style backbone. Learnable matrices $W^Q, W^K, W^V \in \mathbb{R}^{n_{\text{embd}}\times n_{\text{embd}}}$ map hidden states $\mathbf{h}_i$ to queries, keys, and values,

$$\mathbf{q}_i = W^Q\mathbf{h}_i, \qquad \mathbf{k}_i = W^K\mathbf{h}_i, \qquad \mathbf{v}_i = W^V\mathbf{h}_i. \tag{4.1}$$

Heads split embeddings so that $\mathbf{q} = [\mathbf{q}_1; \ldots; \mathbf{q}_{n_h}]$ (and similarly $\mathbf{k}, \mathbf{v}$) with per-head dimension d_h. In head i,

$$\mathbf{o}_{t,i} = \sum_{j=1}^{t} \text{Softmax}\left(\frac{\mathbf{q}_{t,i}^\top\mathbf{k}_{j,i}}{\sqrt{d_h}}\right)\mathbf{v}_{j,i}, \tag{4.2}$$

and $\mathbf{u}_t = W^O[\mathbf{o}_{t,1}; \ldots; \mathbf{o}_{t,n_h}]$ for an output projection W^O.

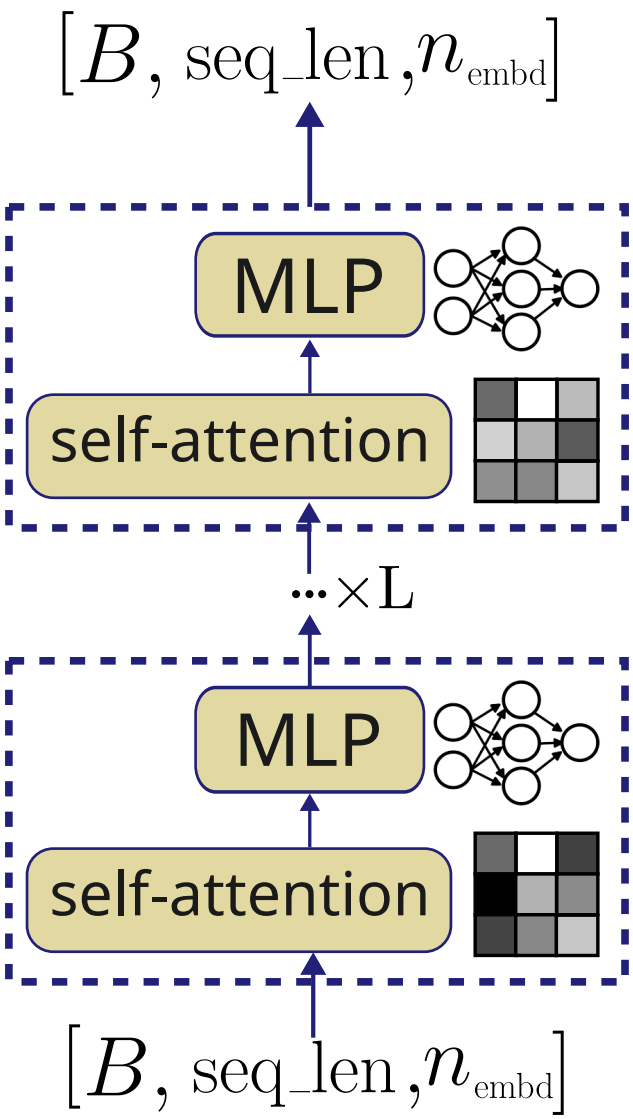


Figure 4.1: Transformer-style backbone used to build electron-wise representations before antisymmetrization.

5. Psiformer ansatz and architecture. The trial wavefunction is an *ansatz*: a model chosen by design and optimized by VMC. We require antisymmetry under particle exchange and expressiveness for strong electron–correlation effects. Following [1, 2], we use a Slater–Jastrow form,

$$\Psi_\theta(\mathbf{R}) = \underbrace{\exp\big(\mathcal{J}_\theta(\mathbf{R})\big)}_{\text{Coulomb correlations}} \times \underbrace{\sum_k \omega_k \det\big[\phi^k_\theta(\mathbf{R})\big]}_{\text{antisymmetry}} . \tag{5.1}$$

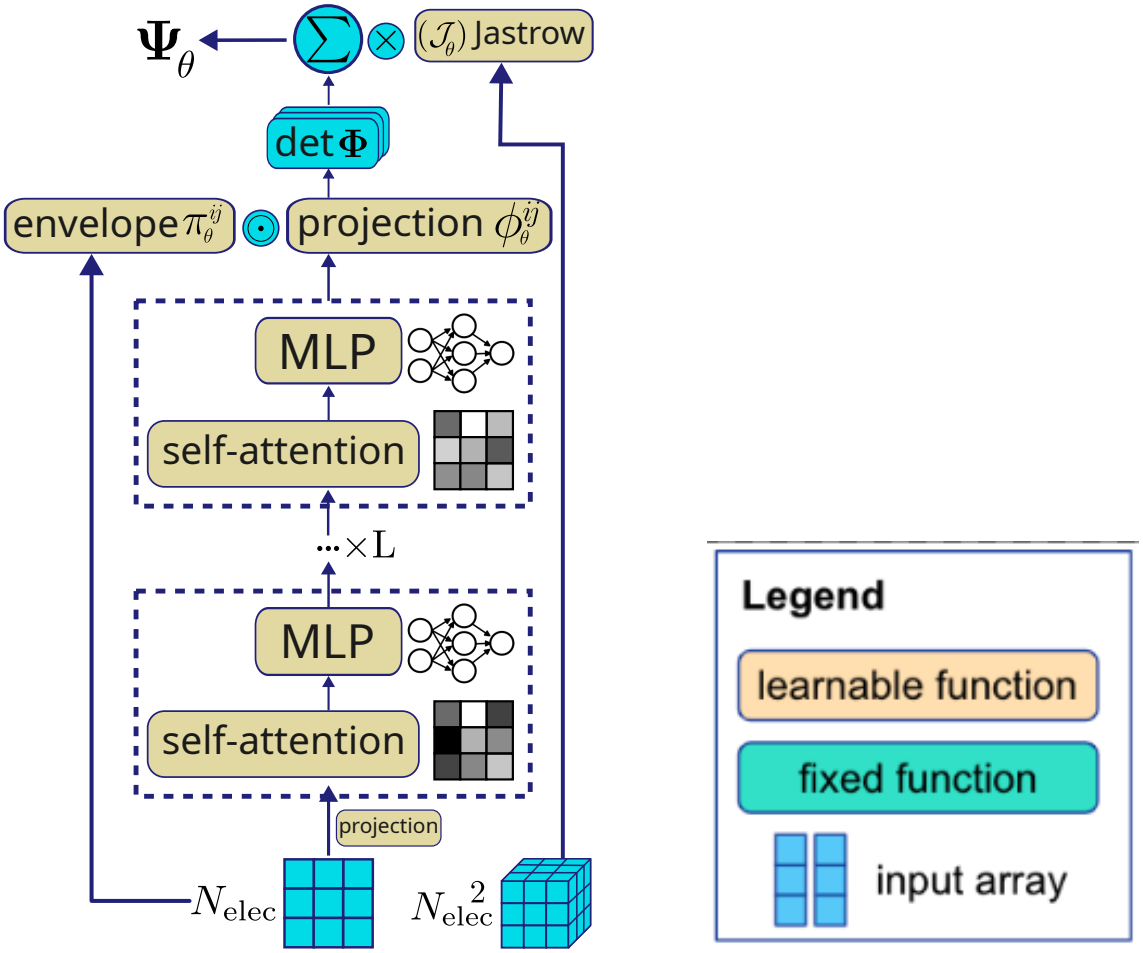
The envelope is implemented as a matrix with learned exponential decay; the Jastrow uses few parameters and is trained in log scale in practice. Figure 5.2 illustrates shape handling in the implementation.

6. Implementation details.

6.1. Hyperparameters and training objective. Figure 6.1a summarizes hyperparameter settings for small and large model variants; Figures 6.1b and 6.1c illustrate the optimization goal and training setup.

6.2. Laplacian via automatic differentiation. The kinetic term requires $\nabla^2\Psi(\mathbf{R}) = \sum_i \partial^2\Psi/\partial R_i^2$. In PyTorch, repeated `autograd.grad` calls accumulate second derivatives:

```
R = R_o.requires_grad_(True)      # particle coordinates
psi = model(R)                    # neural wavefunction
grad_psi = torch.autograd.grad(
    psi, R,
    create_graph=True,
```



(a) Overall Psi Former architecture.

(b) Legend for architecture diagram.

Figure 5.1: Psiformer-style architecture and diagram legend.

```
)[0]
laplacian = 0.0
for i in range(R.shape[-1]):
    laplacian += torch.autograd.grad(
        grad_psi[..., i], R,
        retain_graph=True
    )[0][..., i]
```

6.3. Stable $\log\det$ **gradient.** Since $\Psi_\theta \propto \det \mathbf{\Phi}_\theta(\mathbf{R})$, training uses $\log|\det \mathbf{\Phi}|$ for stability. The matrix derivative identity $\partial(\det A)/\partial A = \det(A)A^{-\mathsf{T}}$ becomes ill-conditioned near singular $\mathbf{\Phi}$. An SVD-based custom autograd op implements $\nabla_A \log|\det A|$ without explicitly forming unstable inverses:

```
import torch

class StableLogDet(torch.autograd.Function):
    @staticmethod
    def forward(ctx, A):
        U, S, Vh = torch.linalg.svd(A)
        ctx.save_for_backward(U, S, Vh)
        return S.log().sum()

    @staticmethod
    def backward(ctx, g):
        U, S, Vh = ctx.saved_tensors
        inv_S = torch.diag_embed(1.0 / S)
        grad_A = U @ inv_S @ Vh
        return g * grad_A
```

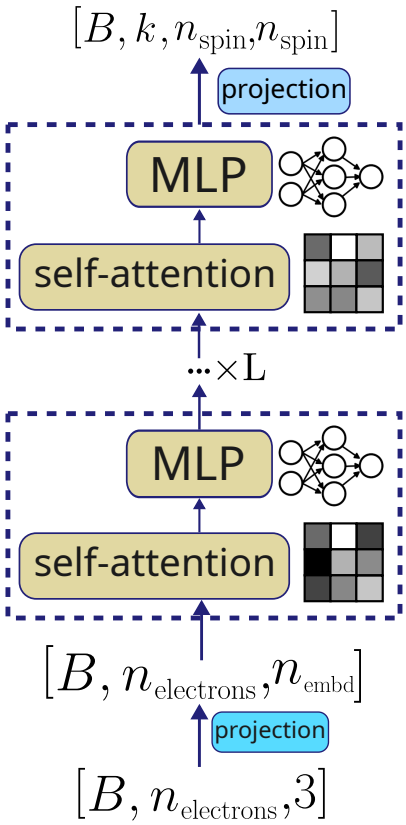


Figure 5.2: Tensor shape flow in the Psiformer implementation.

6.4. Optimizer and batching. AdamW decouples weight decay from the adaptive gradient scaling of Adam. With λ a weight-decay coefficient and η the learning rate,

$$g_t = \nabla\mathcal{L} + \lambda\theta_t, \qquad \theta_{t+1} = \theta_t - \mathrm{AdamStep}(g_t) \qquad \text{(Adam)},$$
$$g_t = \nabla\mathcal{L}, \qquad \theta_{t+1} = \theta_t - \mathrm{AdamStep}(g_t) - \eta\lambda\theta_t \qquad \text{(AdamW)}.$$

Naive step-by-step energy evaluation over MCMC samples yielded low GPU occupancy ($\approx 30\%$). Reshaping batches as

$$(\text{mc_steps}, B, n_e, 3) \longrightarrow (\text{mc_steps} \times B, n_e, 3)$$

for batched forward passes improved utilization to roughly 99% in our runs.

7. Results.

8. Discussion and future work. Several directions remain open: pretraining with external data, reducing the cost of Laplacian computation (a training bottleneck), KFAC or other natural-gradient-style optimizers, FlashAttention-style kernels for long-range mixing, transfer across chemical systems, and empirical scaling laws for model size and data.

acknowledgment. The author thanks the Computer Science Faculty for access to GPU resources, in particular an NVIDIA GeForce RTX 4080 Super, which enabled training and evaluation reported here.

References

[1] F. Hermann, Z. Schätzle, and F. Noé, *A Self Attention Ansatz for Ab Initio Quantum Chemistry*, arXiv:2309.12345, 2023.
[2] J. Pfau, J. S. Spencer, A. G. D. G. Matthews, and W. M. C. Foulkes, *Ab Initio Solution of the Many-Electron Schrödinger Equation with Deep Neural Networks*, Physical Review Research, 2, 033429, 2020.
[3] A. Vaswani et al., *Attention Is All You Need*, Advances in Neural Information Processing Systems (NeurIPS), 2017.

Hyperparameter	Small	Large
Layers L	2	4
Heads H	4	8
Model Dim d	256	512
MLP Dim d_{ff}	1024	2048
Determinants K	1	2
MCMC walkers N_w	1024	2048
MCMC steps / iter	10	10
Learning rate	2×10^{-4}	1×10^{-4}
Total parameters	50441	$3M$

(a) Hyperparameters (small vs. large).

Goal: Obtain accurate ground state-energies using an Ansatz created with **Torch** library

(b) Optimization objective.

Training Loop:

Set the spins and electron number

1.Sample configurations $\mathbf{R}_k$

2.Estimate local energy E_L

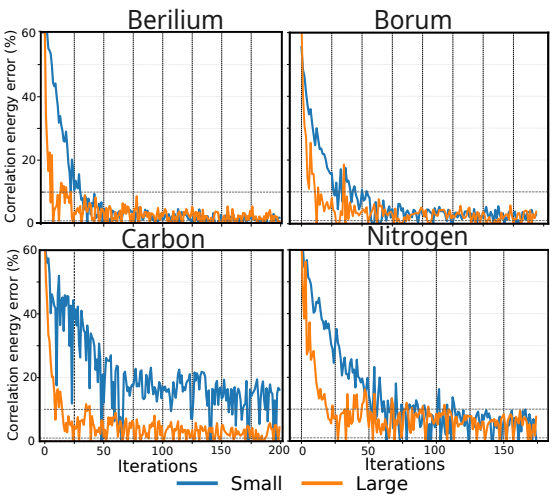
3.Backpropagation using **A.D**

4.Update parameters θ using **AdamGrad**.

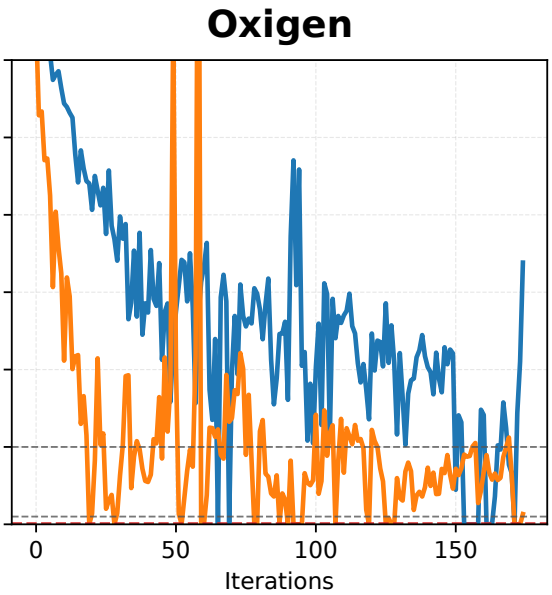
5. Track metrics with **Wandb.**

(c) Training configuration.

Figure 6.1: Methodology: hyperparameters and training.



(a) Convergence curves.



(b) Oxygen convergence.

Figure 7.1: Representative VMC convergence during training.

Atom	E_b	E_s	E_l	Δ_s	Δ_l	Δ_{l-s}
H	-0.500	-0.492	**-0.498**	0.008	0.002	**-0.006**
He	-2.903	-2.801	**-2.893**	0.102	0.010	**-0.092**
Li	-7.478	-7.097	**-7.243**	0.381	0.235	**-0.146**
Be	-14.667	-13.901	**-14.237**	0.766	0.430	**-0.336**
B	-24.653	-24.042	**-24.567**	0.611	0.086	**-0.525**
C	-37.845	-35.492	**-36.457**	2.353	1.388	**-0.965**
N	-54.589	-50.492	**-51.700**	4.097	2.889	**-1.208**
O	-75.067	-63.492	**-72.139**	11.575	2.928	**-8.647**

Figure 7.2: Ground-state energies (Ha): baseline vs. Psiformer-style model for hydrogen–oxygen benchmarks.

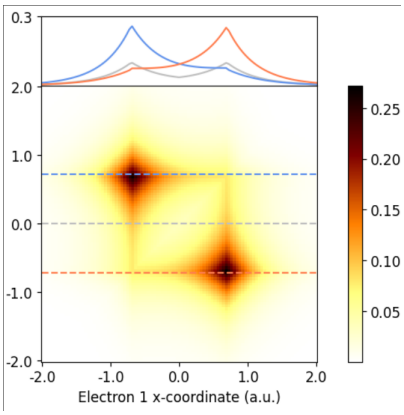


Figure 8.1: Illustrative molecular system (H_2).